

# NEW UNIVERSE

*Microtasks для разработки MVP*

— Phase 0 + Phase 1 в деталях, Phase 2-3 как outline —

**Всего задач: 53**

Эпиков: 13

Документ для планирования + ввод задач для исполняющих моделей

# 1. Как пользоваться этим документом

Документ построен из 3 уровней:

- ГЛОБАЛЬНЫЙ SYSTEM PROMPT (§2) — добавляется в начало контекста ЛЮБОЙ исполняющей модели на КАЖДОЙ задаче. Описывает проект, стек, конвенции.
- ШАБЛОН per-task промпта (§3) — формат подставляемой инструкции под конкретную задачу.
- ЗАДАЧИ (§5+) — список из ~50 микротасок с готовым per-task промptom для копипасты.

Связанные артефакты:

- tasks.csv — импорт в любую систему (Linear, Asana, GitHub Issues).
- tasks.json — машиночитаемый формат, удобен для скриптов.
- import\_to\_github.sh — turn-key скрипт для bulk-создания issue-в в GitHub Project (требуется gh CLI и SSH-ключ).
- README\_github\_import.md — пошаговая инструкция по запуску скрипта.

## 2. Глобальный system prompt

Этот промпт ВСЕГДА вставляется в начале контекста любой исполняющей модели. Он не повторяется в задачах — экономит токены.

Ты — инженер, реализующий задачу для проекта New Universe.

### КОНТЕКСТ ПРОЕКТА

New Universe — это idle/multiplayer space-mining игра, реализуемая как Telegram Mini App. Игрок начинает с одной неприкосновенной "домашней системы" со стартовой планетой, добывает ресурсы, крафтит здания и корабли, исследует планеты в своей системе, затем (через спец. технологию) выходит в общий пул систем (Common Pool), где может колонизировать планеты, торговать и воевать с другими игроками.

Документы, на которые опирайся:

1. Stellar\_Forge\_GDD.docx — основной GDD (механики, баланс, ресурсы, корабли, экономика).
2. New\_Universe\_GDD\_Addendum\_v1.1.docx — изменения (per-player генерация мира, неприкосновенная Home System, ice⇌water конвертация, межсистемные перелёты).
3. Stellar\_Forge\_Infra\_Costs.docx — бюджет инфраструктуры (для понимания масштаба).
4. dev/README.md — описание локального dev-окружения через Docker.

### ТЕХНИЧЕСКИЙ СТЕК

- Backend: Node.js 20 + Fastify + TypeScript + Drizzle ORM + Postgres 16 + Redis + BullMQ
- Frontend: React 18 + TypeScript + Vite + @telegram-apps/sdk-react + Tailwind + PixiJS (для карт)
- Auth: Telegram initData (HMAC валидация)
- Локально: docker compose (см. dev/docker-compose.yml)

### КОНВЕНЦИИ КОДА

- TypeScript strict mode везде. Никаких any (только в крайних случаях с // FIXME).
- Все API-входы валидируются через Zod схемы.
- Все игровые транзакции (списание/начисление ресурсов) — атомарные, в одной БД-транзакции, с SELECT FOR UPDATE. Никаких race conditions.
- Имена БД-таблиц: snake\_case. Имена в TS-коде: camelCase.
- Файлы организованы по feature, не по типу. Например: src/features/buildings/{routes,service,db,tests}.ts
- Любая фоновая работа — через BullMQ, не через setInterval.

### КАК ОТВЕЧАТЬ

1. Прочитай задачу полностью, затем acceptance criteria.
2. Если что-то непонятно — задавай вопрос ДО написания кода.
3. Создавай только файлы, перечисленные в "Files to touch".
4. После реализации напиши краткий список изменений (по 1 строке на файл).
5. Каждая фича должна сопровождаться тестом (unit или integration).

### БЕЗОПАСНОСТЬ

- Никогда не доверять клиенту. Все валидации — на сервере.
- Все Telegram-запросы валидировать по HMAC initData.
- Не логировать секреты, токены, пароли.
- Rate-limit на каждом публичном эндпоинте: 30 req/min на user.

### КОГДА КОММИТИТЬ

- Один коммит = одна задача. Сообщение: "feat(P1-101): create users table".
- Перед коммитом: pnpm lint && pnpm test проходят локально.

- Pull request на main с описанием изменений и ссылкой на задачу.

### 3. Шаблон per-task промпта

Подставляется ПОСЛЕ глобального system prompt. Поля {{...}} заполняются из конкретной задачи.

```
[TASK {{id}}] {{title}}
```

```
ОПИСАНИЕ  
{{description}}
```

```
АССЕРТАЦИЯ КРИТЕРИИ  
{{acceptance_criteria}}
```

```
ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ  
{{files}}
```

```
ЗАВИСИМОСТИ (должны быть выполнены ранее)  
{{deps}}
```

```
КАК ПРОВЕРИТЬ  
{{how_to_verify}}
```

```
ТЕХНИЧЕСКИЕ ЗАМЕТКИ  
{{notes}}
```

## 4. Эпики и общая карта

Всего 13 эпиков, разбитых по фазам.

| Эпик ID       | Фаза | Название                                          | Задач |
|---------------|------|---------------------------------------------------|-------|
| EPIC-P0-INFRA | P0   | Phase 0 — Pre-production: репозиторий, Docker, CI | 10    |
| EPIC-P1-DB    | P1   | Phase 1 — Схема БД и миграции                     | 8     |
| EPIC-P1-AUTH  | P1   | Phase 1 — Авторизация через Telegram              | 3     |
| EPIC-P1-WORLD | P1   | Phase 1 — Генерация мира и Home System            | 4     |
| EPIC-P1-RES   | P1   | Phase 1 — Ресурсы и экономика                     | 3     |
| EPIC-P1-BLD   | P1   | Phase 1 — Здания и постройка                      | 3     |
| EPIC-P1-SHP   | P1   | Phase 1 — Корабли и крафт                         | 2     |
| EPIC-P1-EXP   | P1   | Phase 1 — Экспедиции и видимость                  | 3     |
| EPIC-P1-FE    | P1   | Phase 1 — Frontend (TMA)                          | 7     |
| EPIC-P1-BOT   | P1   | Phase 1 — Telegram-бот и пуши                     | 2     |
| EPIC-P1-QA    | P1   | Phase 1 — Тесты и QA                              | 2     |
| EPIC-P2-OUT   | P2   | Phase 2 — Outline (Expansion)                     | 3     |
| EPIC-P3-OUT   | P3   | Phase 3 — Outline (Multiplayer)                   | 3     |

## 5. Phase 0 — Pre-production: репозиторий, Docker, CI

Эпик: EPIC-P0-INFRA · Задач: 10

### 5.1. [P0-001] Инициализация git-репозитория и базовых файлов

|             |               |
|-------------|---------------|
| ID          | P0-001        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | S             |
| Зависимости | —             |

#### Описание

Создать репозиторий new-universe на GitHub, добавить README, LICENSE (MIT), .gitignore (Node + Docker), .editorconfig, базовый .github/CODEOWNERS.

#### Acceptance criteria

- Репозиторий доступен по <https://github.com/anry88/new-universe>
- README содержит описание проекта в 3-5 предложений и ссылку на GDD
- .gitignore исключает node\_modules, .env, dist, .DS\_Store
- Главная ветка main, защищена от прямых пушей

#### Файлы для создания / изменения

- README.md
- LICENSE
- .gitignore
- .editorconfig
- .github/CODEOWNERS

#### Как проверить

git clone <https://github.com/anry88/new-universe> и `ls -la` показывает все файлы

#### Примечания

GDD-документы скопировать в /docs из текущей папки New Universe.

#### Готовый per-task prompt (копипаста)

|                                                              |
|--------------------------------------------------------------|
| [TASK P0-001] Инициализация git-репозитория и базовых файлов |
| ОПИСАНИЕ                                                     |

Создать репозиторий new-universe на GitHub, добавить README, LICENSE (MIT), .gitignore (Node + Docker), .editorconfig, базовый .github/CODEOWNERS.

#### ACCEPTANCE CRITERIA

1. Репозиторий доступен по <https://github.com/anry88/new-universe>
2. README содержит описание проекта в 3-5 предложений и ссылку на GDD
3. .gitignore исключает node\_modules, .env, dist, .DS\_Store
4. Главная ветка main, защищена от прямых пушей

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- README.md
- LICENSE
- .gitignore
- .editorconfig
- .github/CODEOWNERS

ЗАВИСИМОСТИ (должны быть выполнены ранее)  
(нет)

#### КАК ПРОВЕРИТЬ

git clone <https://github.com/anry88/new-universe> и `ls -la` показывает все файлы

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

GDD-документы скопировать в /docs из текущей папки New Universe.

## 5.2. [P0-002] Docker compose локальное dev-окружение

|             |               |
|-------------|---------------|
| ID          | P0-002        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | M             |
| Зависимости | P0-001        |

### Описание

Перенести dev/docker-compose.yml, dev/.env.example, dev/backend/Dockerfile, dev/frontend/Dockerfile, dev/README.md из текущей папки в репозиторий. Проверить, что docker compose up -d поднимает postgres + redis (backend и frontend будут добавлены позже).

### Acceptance criteria

- ``docker compose up -d postgres redis`` запускается без ошибок
- ``docker compose logs postgres`` показывает healthy state
- ``docker compose exec postgres psql -U nu -d new_universe -c "SELECT 1"`` отвечает 1

### Файлы для создания / изменения

- docker-compose.yml
- .env.example
- README.md (в корне)

### Как проверить

Команды из dev/README.md работают

### Примечания

Порт 5432 не должен конфликтовать с локально установленным Postgres (если есть — поменять порт в .env).

### Готовый per-task prompt (копиаста)

[TASK P0-002] Docker compose локальное dev-окружение

#### ОПИСАНИЕ

Перенести dev/docker-compose.yml, dev/.env.example, dev/backend/Dockerfile, dev/frontend/Dockerfile, dev/README.md из текущей папки в репозиторий. Проверить, что docker compose up -d поднимает postgres + redis (backend и frontend будут добавлены позже).

#### ACCEPTANCE CRITERIA

1. ``docker compose up -d postgres redis`` запускается без ошибок
2. ``docker compose logs postgres`` показывает healthy state
3. ``docker compose exec postgres psql -U nu -d new_universe -c "SELECT 1"`` отвечает 1

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- docker-compose.yml
- .env.example
- README.md (в корне)

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-001

#### КАК ПРОВЕРИТЬ

Команды из dev/README.md работают

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Порт 5432 не должен конфликтовать с локально установленным Postgres (если есть — поменять порт в .env).

### 5.3. [P0-003] Backend skeleton — Fastify + TS

|             |               |
|-------------|---------------|
| ID          | P0-003        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | M             |
| Зависимости | P0-002        |

#### Описание

Создать backend/ с минимальным Fastify-приложением: GET /health, структурированное логирование через Pino, чтение env через dotenv + Zod-валидация. Запуск через docker compose up backend.

#### Acceptance criteria

- GET /health возвращает {"status":"ok","ts":...}
- Логи в JSON-формате, в dev — pino-pretty
- Сервер падает на старте, если в .env отсутствует обязательная переменная

#### Файлы для создания / изменения

- backend/src/index.ts
- backend/src/lib/env.ts
- backend/src/lib/logger.ts
- backend/src/routes/health.ts
- backend/tsconfig.json

#### Как проверить

curl http://localhost:3000/health → 200 ok

#### Примечания

*Fastify v5, TypeScript strict, ES modules.*

#### Готовый per-task prompt (копипаста)

[TASK P0-003] Backend skeleton — Fastify + TS

##### ОПИСАНИЕ

Создать backend/ с минимальным Fastify-приложением: GET /health, структурированное логирование через Pino, чтение env через dotenv + Zod-валидация. Запуск через docker compose up backend.

##### ACCEPTANCE CRITERIA

1. GET /health возвращает {"status":"ok","ts":...}
2. Логи в JSON-формате, в dev — pino-pretty

3. Сервер падает на старте, если в .env отсутствует обязательная переменная

**ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ**

- backend/src/index.ts
- backend/src/lib/env.ts
- backend/src/lib/logger.ts
- backend/src/routes/health.ts
- backend/tsconfig.json

**ЗАВИСИМОСТИ (должны быть выполнены ранее)**

- P0-002

**КАК ПРОВЕРИТЬ**

curl http://localhost:3000/health → 200 ok

**ТЕХНИЧЕСКИЕ ЗАМЕТКИ**

Fastify v5, TypeScript strict, ES modules.

## 5.4. [P0-004] Frontend skeleton — Vite + React + TMA SDK

|             |               |
|-------------|---------------|
| ID          | P0-004        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | M             |
| Зависимости | P0-002        |

### Описание

Создать frontend/ с минимальным React-приложением, инициализацией Telegram WebApp SDK, Tailwind CSS, страницей "Hello, {username}". Запуск через docker compose up frontend.

### Acceptance criteria

- `http://localhost:5173` отдаёт страницу с приветствием
- При запуске внутри Telegram — берёт username из `initData`
- Поддерживается dark/light тема Telegram
- Tailwind CSS работает (классы применяются)

### Файлы для создания / изменения

- frontend/src/main.tsx
- frontend/src/App.tsx
- frontend/index.html
- frontend/vite.config.ts
- frontend/tailwind.config.ts
- frontend/src/index.css

### Как проверить

`docker compose up frontend` → открыть `http://localhost:5173` → отображается страница

### Примечания

Использовать `@telegram-apps/sdk-react`. Если запущено НЕ из Telegram — использовать заглушку `user="DevUser"`.

### Готовый per-task prompt (копипаста)

[TASK P0-004] Frontend skeleton — Vite + React + TMA SDK

#### ОПИСАНИЕ

Создать frontend/ с минимальным React-приложением, инициализацией Telegram WebApp SDK, Tailwind CSS, страницей "Hello, {username}". Запуск через `docker compose up frontend`.

#### ACCEPTANCE CRITERIA

1. `http://localhost:5173` отдаёт страницу с приветствием
2. При запуске внутри Telegram — берёт username из `initData`
3. Поддерживается dark/light тема Telegram
4. Tailwind CSS работает (классы применяются)

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- `frontend/src/main.tsx`
- `frontend/src/App.tsx`
- `frontend/index.html`
- `frontend/vite.config.ts`
- `frontend/tailwind.config.ts`
- `frontend/src/index.css`

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-002

#### КАК ПРОВЕРИТЬ

`docker compose up frontend` → открыть `http://localhost:5173` → отображается страница

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать `@telegram-apps/sdk-react`. Если запущено НЕ из Telegram — использовать заглушку `user="DevUser"`.

## 5.5. [P0-005] Drizzle ORM + миграции

|             |               |
|-------------|---------------|
| ID          | P0-005        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | S             |
| Зависимости | P0-003        |

### Описание

Установить drizzle-orm + drizzle-kit. Настроить drizzle.config.ts. Создать пустую schema.ts. Команды `npm run db:migrate`, `db:generate`, `db:studio` должны работать.

### Acceptance criteria

- ``npm run db:generate`` создаёт миграцию из изменений `schema.ts`
- ``npm run db:migrate`` накатывает миграции на dev БД
- ``npm run db:studio`` запускает Drizzle Studio на `http://localhost:4983`

### Файлы для создания / изменения

- `backend/drizzle.config.ts`
- `backend/src/db/index.ts`
- `backend/src/db/schema.ts`
- `backend/src/db/migrations/.gitkeep`

### Как проверить

``npm run db:migrate`` без ошибок, ``dt`` в psql показывает только пустые системные таблицы

### Готовый per-task prompt (копипаста)

[TASK P0-005] Drizzle ORM + миграции

#### ОПИСАНИЕ

Установить drizzle-orm + drizzle-kit. Настроить drizzle.config.ts. Создать пустую schema.ts. Команды `npm run db:migrate`, `db:generate`, `db:studio` должны работать.

#### ACCEPTANCE CRITERIA

1. ``npm run db:generate`` создаёт миграцию из изменений `schema.ts`
2. ``npm run db:migrate`` накатывает миграции на dev БД
3. ``npm run db:studio`` запускает Drizzle Studio на `http://localhost:4983`

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- `backend/drizzle.config.ts`
- `backend/src/db/index.ts`
- `backend/src/db/schema.ts`
- `backend/src/db/migrations/.gitkeep`

ЗАВИСИМОСТИ (должны быть выполнены ранее)  
- P0-003

КАК ПРОВЕРИТЬ

`npm run db:migrate` без ошибок, `dt` в psql показывает только пустые системные таблицы

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 5.6. [P0-006] GitHub Actions CI

|             |                |
|-------------|----------------|
| ID          | P0-006         |
| Эпик        | EPIC-P0-INFRA  |
| Размер      | M              |
| Зависимости | P0-003, P0-004 |

### Описание

Workflow `.github/workflows/ci.yml`: на каждый PR — lint, type-check, тесты для backend и frontend. Использовать docker compose для прогона. Кэшировать node\_modules.

### Acceptance criteria

- PR на main → запускается workflow
- Workflow проходит на ~3-5 минут
- Падение тестов блокирует merge

### Файлы для создания / изменения

- `.github/workflows/ci.yml`

### Как проверить

Создать PR с заведомо проходящими тестами → CI зелёный

### Примечания

Использовать `pnpm` если в проекте `pnpm`, иначе `npm`. Кэширование через `actions/cache`.

### Готовый per-task prompt (копипаста)

[TASK P0-006] GitHub Actions CI

#### ОПИСАНИЕ

Workflow `.github/workflows/ci.yml`: на каждый PR — lint, type-check, тесты для backend и frontend. Использовать docker compose для прогона. Кэшировать node\_modules.

#### ACCEPTANCE CRITERIA

1. PR на main → запускается workflow
2. Workflow проходит на ~3-5 минут
3. Падение тестов блокирует merge

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- `.github/workflows/ci.yml`

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-003
- P0-004

#### КАК ПРОВЕРИТЬ

Создать PR с заведомо проходящими тестами → CI зелёный

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать `pnpm` если в проекте `pnpm`, иначе `npm`. Кэширование через `actions/cache`.

## 5.7. [P0-007] Sentry интеграция

|             |                |
|-------------|----------------|
| ID          | P0-007         |
| Эпик        | EPIC-P0-INFRA  |
| Размер      | S              |
| Зависимости | P0-003, P0-004 |

### Описание

Создать 2 проекта в Sentry (free tier): new-universe-backend, new-universe-tma. Подключить в код через @sentry/node и @sentry/react. DSN в .env.

### Acceptance criteria

- Тестовая ошибка на бэкенде попадает в Sentry
- Тестовая ошибка на фронте попадает в Sentry
- В Sentry привязаны git releases

### Файлы для создания / изменения

- backend/src/lib/sentry.ts
- frontend/src/lib/sentry.ts

### Как проверить

GET /debug-sentry → видно ошибку в Sentry UI

### Примечания

Эндпоинт /debug-sentry убрать после тестирования.

### Готовый per-task prompt (копипаста)

[TASK P0-007] Sentry интеграция

#### ОПИСАНИЕ

Создать 2 проекта в Sentry (free tier): new-universe-backend, new-universe-tma. Подключить в код через @sentry/node и @sentry/react. DSN в .env.

#### ACCEPTANCE CRITERIA

1. Тестовая ошибка на бэкенде попадает в Sentry
2. Тестовая ошибка на фронте попадает в Sentry
3. В Sentry привязаны git releases

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/lib/sentry.ts
- frontend/src/lib/sentry.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-003
- P0-004

КАК ПРОВЕРИТЬ

GET /debug-sentry → видно ошибку в Sentry UI

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Эндпоинт /debug-sentry убрать после тестирования.

## 5.8. [P0-008] Структурированное логирование

|             |               |
|-------------|---------------|
| ID          | P0-008        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | S             |
| Зависимости | P0-003        |

### Описание

Все логи бэкенда — через Pino, в JSON. Поля: ts, level, msg, requestId, userId (если есть), feature.

### Acceptance criteria

- Каждый HTTP-запрос логируется со своим requestId (генерируется через uuid v4)
- Логи в dev — через pino-pretty, читаемые в терминале
- Логи в проде — pure JSON, готовы для BetterStack/Logtail

### Файлы для создания / изменения

- backend/src/lib/logger.ts
- backend/src/middleware/request-id.ts

### Как проверить

Сделать запрос → в логах видно requestId

### Готовый per-task prompt (копипаста)

[TASK P0-008] Структурированное логирование

#### ОПИСАНИЕ

Все логи бэкенда — через Pino, в JSON. Поля: ts, level, msg, requestId, userId (если есть), feature.

#### ACCEPTANCE CRITERIA

1. Каждый HTTP-запрос логируется со своим requestId (генерируется через uuid v4)
2. Логи в dev — через pino-pretty, читаемые в терминале
3. Логи в проде — pure JSON, готовы для BetterStack/Logtail

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/lib/logger.ts
- backend/src/middleware/request-id.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-003

#### КАК ПРОВЕРИТЬ

Сделать запрос → в логах видно requestId

## ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 5.9. [P0-009] Vitest + первый smoke-тест

|             |               |
|-------------|---------------|
| ID          | P0-009        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | S             |
| Зависимости | P0-003        |

### Описание

Подключить vitest. Написать smoke-тест для GET /health.

### Acceptance criteria

- `npm test` запускает vitest и проходит
- Тест проверяет, что /health возвращает 200 и {status:"ok"}

### Файлы для создания / изменения

- backend/vitest.config.ts
- backend/src/routes/health.test.ts

### Как проверить

`docker compose exec backend npm test` зелёный

### Готовый per-task prompt (копипаста)

[TASK P0-009] Vitest + первый smoke-тест

#### ОПИСАНИЕ

Подключить vitest. Написать smoke-тест для GET /health.

#### ACCEPTANCE CRITERIA

1. `npm test` запускает vitest и проходит
2. Тест проверяет, что /health возвращает 200 и {status:"ok"}

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/vitest.config.ts
- backend/src/routes/health.test.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-003

#### КАК ПРОВЕРИТЬ

`docker compose exec backend npm test` зелёный

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 5.10. [P0-010] Telegram-бот через @BotFather + dev webhook

|             |               |
|-------------|---------------|
| ID          | P0-010        |
| Эпик        | EPIC-P0-INFRA |
| Размер      | M             |
| Зависимости | P0-003        |

### Описание

Создать тестового бота через @BotFather (имя NewUniverseDevBot). Получить токен. Настроить dev-webhook через cloudflared tunnel. Документировать процесс в dev/README.md.

### Acceptance criteria

- Бот существует в Telegram
- TELEGRAM\_BOT\_TOKEN записан в личные заметки разработчика (НЕ в репо!)
- Webhook URL настроен через @BotFather → /setwebhook
- Тестовое сообщение боту → попадает в backend log

### Файлы для создания / изменения

- dev/TELEGRAM\_SETUP.md

### Как проверить

Отправить /start боту → видно в логах backend

### Примечания

Описать в TELEGRAM\_SETUP.md шаги: создание бота, получение токена, настройка туннеля. Токен хранить ТОЛЬКО в .env (gitignored).

### Готовый per-task prompt (копипаста)

[TASK P0-010] Telegram-бот через @BotFather + dev webhook

#### ОПИСАНИЕ

Создать тестового бота через @BotFather (имя NewUniverseDevBot). Получить токен. Настроить dev-webhook через cloudflared tunnel. Документировать процесс в dev/README.md.

#### ACCEPTANCE CRITERIA

1. Бот существует в Telegram
2. TELEGRAM\_BOT\_TOKEN записан в личные заметки разработчика (НЕ в репо!)
3. Webhook URL настроен через @BotFather → /setwebhook
4. Тестовое сообщение боту → попадает в backend log

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- dev/TELEGRAM\_SETUP.md

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-003

#### КАК ПРОВЕРИТЬ

Отправить /start боту → видно в логах backend

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Описать в TELEGRAM\_SETUP.md шаги: создание бота, получение токена, настройка туннеля. Токен хранить ТОЛЬКО в .env (gitignored).

## 6. Phase 1 — Схема БД и миграции

Эпик: EPIC-P1-DB · Задач: 8

### 6.1. [P1-101] Таблица users

|             |            |
|-------------|------------|
| ID          | P1-101     |
| Эпик        | EPIC-P1-DB |
| Размер      | S          |
| Зависимости | P0-005     |

#### Описание

Создать таблицу users в schema.ts: id (uuid), tg\_id (bigint unique), tg\_username, tg\_first\_name, created\_at, premium\_until, power\_score (default 0). Сгенерировать и применить миграцию.

#### Acceptance criteria

- Таблица users создана со всеми полями
- Индекс по tg\_id (unique)
- Миграция в backend/src/db/migrations/

#### Файлы для создания / изменения

- backend/src/db/schema/users.ts
- backend/src/db/migrations/0001\_users.sql (auto-generated)

#### Как проверить

`\d users` в psql показывает все поля

#### Примечания

Использовать drizzle pgTable. Импортировать в src/db/schema.ts.

#### Готовый per-task prompt (копипаста)

[TASK P1-101] Таблица users

##### ОПИСАНИЕ

Создать таблицу users в schema.ts: id (uuid), tg\_id (bigint unique), tg\_username, tg\_first\_name, created\_at, premium\_until, power\_score (default 0). Сгенерировать и применить миграцию.

##### ACCEPTANCE CRITERIA

1. Таблица users создана со всеми полями
2. Индекс по tg\_id (unique)

### 3. Миграция в backend/src/db/migrations/

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/users.ts
- backend/src/db/migrations/0001\_users.sql (auto-generated)

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-005

#### КАК ПРОВЕРИТЬ

`\d users` в psql показывает все поля

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать drizzle pgTable. Импортировать в src/db/schema.ts.

## 6.2. [P1-102] Каталог ресурсов: таблица resources + сидер

|             |            |
|-------------|------------|
| ID          | P1-102     |
| Эпик        | EPIC-P1-DB |
| Размер      | S          |
| Зависимости | P1-101     |

### Описание

Таблица resources: id (slug, primary key), name (json: ru/en), tier (1-4), symbol, base\_regen\_rate, default\_storage\_cap. Сидер заполняет 13 ресурсов (см. addendum + GDD).

### Acceptance criteria

- Таблица resources заполнена 13 записями: water, iron, carbon, silicon, methane, copper, aluminum, titanium, ice, mercury, magnesium, lead, tritium
- `npm run db:seed` идиempotentен (можно запускать повторно)
- Каждая запись имеет ru/en название

### Файлы для создания / изменения

- backend/src/db/schema/resources.ts
- backend/src/db/seed/resources.ts

### Как проверить

`SELECT count(\*) FROM resources` → 13

### Примечания

*Tritium — новый ресурс (см. addendum §3.6). Tier 3 минор, нужен для прыжкового топлива.*

### Готовый per-task prompt (копипаста)

[TASK P1-102] Каталог ресурсов: таблица resources + сидер

ОПИСАНИЕ

Таблица resources: id (slug, primary key), name (json: ru/en), tier (1-4), symbol, base\_regen\_rate, default\_storage\_cap. Сидер заполняет 13 ресурсов (см. addendum + GDD).

ACCEPTANCE CRITERIA

1. Таблица resources заполнена 13 записями: water, iron, carbon, silicon, methane, copper, aluminum, titanium, ice, mercury, magnesium, lead, tritium
2. `npm run db:seed` идиempotentен (можно запускать повторно)
3. Каждая запись имеет ru/en название

**ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ**

- backend/src/db/schema/resources.ts
- backend/src/db/seed/resources.ts

**ЗАВИСИМОСТИ (должны быть выполнены ранее)**

- P1-101

**КАК ПРОВЕРИТЬ**

`SELECT count(\*) FROM resources` → 13

**ТЕХНИЧЕСКИЕ ЗАМЕТКИ**

Tritium — новый ресурс (см. addendum §3.6). Tier 3 минор, нужен для прыжкового топлива.

### 6.3. [P1-103] Планеты, planet\_resources, richness

|             |            |
|-------------|------------|
| ID          | P1-103     |
| Эпик        | EPIC-P1-DB |
| Размер      | M          |
| Зависимости | P1-102     |

#### Описание

Таблицы: systems (id, owner\_id nullable, is\_home, sector\_x/y/z, name, seed), planets (id, system\_id, biome, size, slot\_count, name), planet\_resources (planet\_id, resource\_id, amount, last\_update\_at, regen\_rate), richness (planet\_id, resource\_id, value 0..5).

#### Acceptance criteria

- Все 4 таблицы созданы
- Внешние ключи: planets.system\_id → systems.id, planet\_resources/richness → planets/resources
- Индексы: planets.system\_id, planet\_resources(planet\_id, resource\_id) unique

#### Файлы для создания / изменения

- backend/src/db/schema/world.ts
- backend/src/db/migrations/...sql

#### Как проверить

Миграция применена, `id planets` показывает поля

#### Примечания

*is\_home\_system* флаг будет использоваться для блокировки чужих обращений к домашней системе.

#### Готовый per-task prompt (копипаста)

[TASK P1-103] Планеты, planet\_resources, richness

ОПИСАНИЕ

Таблицы: systems (id, owner\_id nullable, is\_home, sector\_x/y/z, name, seed), planets (id, system\_id, biome, size, slot\_count, name), planet\_resources (planet\_id, resource\_id, amount, last\_update\_at, regen\_rate), richness (planet\_id, resource\_id, value 0..5).

ACCEPTANCE CRITERIA

1. Все 4 таблицы созданы
2. Внешние ключи: planets.system\_id → systems.id, planet\_resources/richness → planets/resources
3. Индексы: planets.system\_id, planet\_resources(planet\_id, resource\_id) unique

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/world.ts
- backend/src/db/migrations/...sql

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-102

#### КАК ПРОВЕРИТЬ

Миграция применена, `d planets` показывает поля

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

is\_home\_system флаг будет использоваться для блокировки чужих обращений к домашней системе.

## 6.4. [P1-104] building\_types (catalog) + buildings (instances)

|             |            |
|-------------|------------|
| ID          | P1-104     |
| Эпик        | EPIC-P1-DB |
| Размер      | M          |
| Зависимости | P1-103     |

### Описание

building\_types: id (slug), name (json), category, max\_level, deps (jsonb массив зависимостей), base\_cost (jsonb), base\_time\_sec, base\_output (jsonb), energy\_consumption. buildings: id, planet\_id, type\_id, level, queue\_action (build|upgrade|null), queue\_completes\_at. Сидер заполняет 10 базовых типов из GDD §5.

### Acceptance criteria

- Таблицы созданы
- Сидер заполняет building\_types: command\_center, mine, drill, storage, smelter, spaceport, shipyard, lab, cryo\_factory, solar\_plant
- Стоимости и времена соответствуют GDD

### Файлы для создания / изменения

- backend/src/db/schema/buildings.ts
- backend/src/db/seed/building-types.ts

### Как проверить

`SELECT slug FROM building\_types ORDER BY slug` → 10 строк

### Примечания

*cryo\_factory* — новое здание из addendum (для лёд⇌вода).

### Готовый per-task prompt (копипаста)

[TASK P1-104] building\_types (catalog) + buildings (instances)

#### ОПИСАНИЕ

building\_types: id (slug), name (json), category, max\_level, deps (jsonb массив зависимостей), base\_cost (jsonb), base\_time\_sec, base\_output (jsonb), energy\_consumption.  
buildings: id, planet\_id, type\_id, level, queue\_action (build|upgrade|null), queue\_completes\_at.  
Сидер заполняет 10 базовых типов из GDD §5.

#### ACCEPTANCE CRITERIA

1. Таблицы созданы
2. Сидер заполняет building\_types: command\_center, mine, drill, storage, smelter, spaceport, shipyard, lab, cryo\_factory, solar\_plant

3. Стоимости и времена соответствуют GDD

ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/buildings.ts
- backend/src/db/seed/building-types.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-103

КАК ПРОВЕРИТЬ

`SELECT slug FROM building\_types ORDER BY slug` → 10 строк

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

cryo\_factory — новое здание из addendum (для лёд→вода).

## 6.5. [P1-105] ship\_types + ships

|             |            |
|-------------|------------|
| ID          | P1-105     |
| Эпик        | EPIC-P1-DB |
| Размер      | M          |
| Зависимости | P1-103     |

### Описание

ship\_types: id (slug), name, role, hp, speed, cargo, dps, armor, fuel\_consumption, build\_time\_sec, build\_cost (jsonb), required\_buildings, sensor\_range. ships: id, owner\_id, type\_id, location\_planet\_id (nullable), status (idle|moving|building), cargo\_json, fuel. Сидер заполняет 5 типов: scout, cargo\_light, colonizer, recon\_probe, jump\_ship.

### Acceptance criteria

- Таблицы созданы
- Сидер: 5 ship\_types (Scout, Cargo Light, Colonizer, Recon Probe, Jump Ship)
- Поле sensor\_range заполнено (Scout=30, Recon=60, Cargo=8 и т.д.)

### Файлы для создания / изменения

- backend/src/db/schema/ships.ts
- backend/src/db/seed/ship-types.ts

### Как проверить

```
`SELECT slug, sensor_range FROM ship_types`
```

### Примечания

*Jump Ship — новый тип из addendum, требует Engines lvl 3 + Jump Drive.*

### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-105] ship_types + ships</p> <p>ОПИСАНИЕ</p> <p>ship_types: id (slug), name, role, hp, speed, cargo, dps, armor, fuel_consumption, build_time_sec, build_cost (jsonb), required_buildings, sensor_range.</p> <p>ships: id, owner_id, type_id, location_planet_id (nullable), status (idle moving building), cargo_json, fuel.</p> <p>Сидер заполняет 5 типов: scout, cargo_light, colonizer, recon_probe, jump_ship.</p> <p>ACCEPTANCE CRITERIA</p> <ol style="list-style-type: none"><li>1. Таблицы созданы</li><li>2. Сидер: 5 ship_types (Scout, Cargo Light, Colonizer, Recon Probe, Jump Ship)</li><li>3. Поле sensor_range заполнено (Scout=30, Recon=60, Cargo=8 и т.д.)</li></ol> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/ships.ts
- backend/src/db/seed/ship-types.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-103

#### КАК ПРОВЕРИТЬ

```
`SELECT slug, sensor_range FROM ship_types`
```

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Jump Ship — новый тип из addendum, требует Engines lvl 3 + Jump Drive.

## 6.6. [P1-106] research, research\_progress

|             |            |
|-------------|------------|
| ID          | P1-106     |
| Эпик        | EPIC-P1-DB |
| Размер      | S          |
| Зависимости | P1-101     |

### Описание

research\_branches (catalog: mining, engineering, engines, weapons, sensors, logistics, jump\_drive). research\_progress (user\_id, branch, level, completes\_at).

### Acceptance criteria

- Таблицы созданы
- Сидер заполняет 7 веток (включая jump\_drive из addendum)

### Файлы для создания / изменения

- backend/src/db/schema/research.ts
- backend/src/db/seed/research-branches.ts

### Как проверить

```
`SELECT * FROM research_branches`
```

### Готовый per-task prompt (копипаста)

[TASK P1-106] research, research\_progress

#### ОПИСАНИЕ

research\_branches (catalog: mining, engineering, engines, weapons, sensors, logistics, jump\_drive). research\_progress (user\_id, branch, level, completes\_at).

#### ACCEPTANCE CRITERIA

1. Таблицы созданы
2. Сидер заполняет 7 веток (включая jump\_drive из addendum)

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/research.ts
- backend/src/db/seed/research-branches.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-101

#### КАК ПРОВЕРИТЬ

```
`SELECT * FROM research_branches`
```

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ



## 6.7. [P1-107] expeditions

|                    |            |
|--------------------|------------|
| <b>ID</b>          | P1-107     |
| <b>Эпик</b>        | EPIC-P1-DB |
| <b>Размер</b>      | M          |
| <b>Зависимости</b> | P1-105     |

### Описание

expeditions: id, ship\_id, type (scout|cargo|colonize|jump), origin\_planet\_id, target\_x/y/z, target\_planet\_id (nullable), status (queued|in\_flight|arrived|returning|completed|failed), eta, returned\_at, result (jsonb).

### Acceptance criteria

- Таблица создана
- Индексы по eta + status (для быстрого poll-а воркером)

### Файлы для создания / изменения

- backend/src/db/schema/expeditions.ts

### Как проверить

Миграция применена

### Готовый per-task prompt (копипаста)

[TASK P1-107] expeditions

#### ОПИСАНИЕ

expeditions: id, ship\_id, type (scout|cargo|colonize|jump), origin\_planet\_id, target\_x/y/z, target\_planet\_id (nullable), status (queued|in\_flight|arrived|returning|completed|failed), eta, returned\_at, result (jsonb).

#### ACCEPTANCE CRITERIA

1. Таблица создана
2. Индексы по eta + status (для быстрого poll-а воркером)

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/expeditions.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-105

#### КАК ПРОВЕРИТЬ

Миграция применена

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ



## 6.8. [P1-108] discovered\_planets, discovered\_systems (per-player)

|             |            |
|-------------|------------|
| ID          | P1-108     |
| Эпик        | EPIC-P1-DB |
| Размер      | S          |
| Зависимости | P1-103     |

### Описание

Таблицы для per-player видимости (см. addendum §3.4). discovered\_planets (user\_id, planet\_id, discovered\_at). discovered\_systems (user\_id, system\_id, discovered\_at). Уникальный индекс (user\_id, planet\_id) / (user\_id, system\_id).

### Acceptance criteria

- Таблицы созданы
- Индексы корректные

### Файлы для создания / изменения

- backend/src/db/schema/discovery.ts

### Как проверить

Миграция применена

### Примечания

*Все API, возвращающие планеты/системы, должны фильтровать через JOIN с этими таблицами.*

### Готовый per-task prompt (копипаста)

[TASK P1-108] discovered\_planets, discovered\_systems (per-player)

#### ОПИСАНИЕ

Таблицы для per-player видимости (см. addendum §3.4). discovered\_planets (user\_id, planet\_id, discovered\_at). discovered\_systems (user\_id, system\_id, discovered\_at). Уникальный индекс (user\_id, planet\_id) / (user\_id, system\_id).

#### АССЕРТАНЦЕ CRITERIA

1. Таблицы созданы
2. Индексы корректные

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/discovery.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-103

КАК ПРОВЕРИТЬ  
Миграция применена

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Все API, возвращающие планеты/системы, должны фильтровать через JOIN с этими таблицами.

## 7. Phase 1 — Авторизация через Telegram

Эпик: EPIC-P1-AUTH · Задач: 3

### 7.1. [P1-120] Telegram initData валидация

|             |              |
|-------------|--------------|
| ID          | P1-120       |
| Эпик        | EPIC-P1-AUTH |
| Размер      | M            |
| Зависимости | P0-008       |

#### Описание

Middleware, проверяющий заголовок X-Telegram-Init-Data: HMAC по boilerplate Telegram. Парсит user\_id, username из строки. Если валидно — кладёт в request.user.

#### Acceptance criteria

- Корректный initData (от реального Telegram) → middleware пропускает
- Истёкший initData (auth\_date > 1 час) → 401
- Подделанный hash → 401
- Без заголовка → 401

#### Файлы для создания / изменения

- backend/src/middleware/telegram-auth.ts
- backend/src/lib/telegram.ts
- backend/src/middleware/telegram-auth.test.ts

#### Как проверить

Unit-тест проходит

#### Примечания

Использовать алгоритм из официальной документации:

<https://core.telegram.org/bots/webapps#validating-data-received-via-the-mini-app>

#### Готовый per-task prompt (копипаста)

[TASK P1-120] Telegram initData валидация

##### ОПИСАНИЕ

Middleware, проверяющий заголовок X-Telegram-Init-Data: HMAC по boilerplate Telegram. Парсит user\_id, username из строки. Если валидно — кладёт в request.user.

#### ACCEPTANCE CRITERIA

1. Корректный initData (от реального Telegram) → middleware пропускает
2. Истёкший initData (auth\_date > 1 час) → 401
3. Подделанный hash → 401
4. Без заголовка → 401

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/middleware/telegram-auth.ts
- backend/src/lib/telegram.ts
- backend/src/middleware/telegram-auth.test.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-008

#### КАК ПРОВЕРИТЬ

Unit-тест проходит

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать алгоритм из официальной документации:  
<https://core.telegram.org/bots/webapps#validating-data-received-via-the-mini-app>

## 7.2. [P1-121] POST /auth/telegram → создание/получение пользователя

|             |                |
|-------------|----------------|
| ID          | P1-121         |
| Эпик        | EPIC-P1-AUTH   |
| Размер      | M              |
| Зависимости | P1-120, P1-141 |

### Описание

Эндпоинт принимает валидированный initData (через middleware). Если user с tg\_id не существует — создаёт + триггерит P1-141 (генерация Home System). Возвращает JWT-сессию (httpOnly cookie + body).

### Acceptance criteria

- Первый запрос с новым tg\_id → создаёт user + home system
- Второй запрос с тем же tg\_id → возвращает existing user
- Возвращает session token, который можно использовать в Authorization header

### Файлы для создания / изменения

- backend/src/features/auth/routes.ts
- backend/src/features/auth/service.ts
- backend/src/features/auth/auth.test.ts

### Как проверить

curl с тестовым initData → возвращает 200 + token

### Примечания

JWT secret из .env. Срок жизни 30 дней.

### Готовый per-task prompt (копипаста)

[TASK P1-121] POST /auth/telegram → создание/получение пользователя

#### ОПИСАНИЕ

Эндпоинт принимает валидированный initData (через middleware). Если user с tg\_id не существует — создаёт + триггерит P1-141 (генерация Home System). Возвращает JWT-сессию (httpOnly cookie + body).

#### ACCEPTANCE CRITERIA

1. Первый запрос с новым tg\_id → создаёт user + home system
2. Второй запрос с тем же tg\_id → возвращает existing user
3. Возвращает session token, который можно использовать в Authorization header

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/auth/routes.ts
- backend/src/features/auth/service.ts
- backend/src/features/auth/auth.test.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-120
- P1-141

#### КАК ПРОВЕРИТЬ

curl с тестовым initData → возвращает 200 + token

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

JWT secret из .env. Срок жизни 30 дней.

### 7.3. [P1-122] GET /me → состояние игрока

|             |                |
|-------------|----------------|
| ID          | P1-122         |
| Эпик        | EPIC-P1-AUTH   |
| Размер      | S              |
| Зависимости | P1-121, P1-130 |

#### Описание

Возвращает: текущий user, домашняя система, открытые планеты, корабли, очереди стройки, ресурсы (с актуальным lazy-compute), исследования.

#### Acceptance criteria

- Без авторизации → 401
- С авторизацией → 200 с полным state в JSON
- Время ответа ≤ 200 мс при нормальной нагрузке

#### Файлы для создания / изменения

- backend/src/features/me/routes.ts
- backend/src/features/me/service.ts

#### Как проверить

Интеграционный тест: создать user → GET /me → видеть Home System

#### Примечания

*Это основной "single source of truth" для frontend. Должен возвращать минимально необходимое.*

#### Готовый per-task prompt (копипаста)

[TASK P1-122] GET /me → состояние игрока

##### ОПИСАНИЕ

Возвращает: текущий user, домашняя система, открытые планеты, корабли, очереди стройки, ресурсы (с актуальным lazy-compute), исследования.

##### ACCEPTANCE CRITERIA

1. Без авторизации → 401
2. С авторизацией → 200 с полным state в JSON
3. Время ответа ≤ 200 мс при нормальной нагрузке

##### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/me/routes.ts
- backend/src/features/me/service.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-121
- P1-130

КАК ПРОВЕРИТЬ

Интеграционный тест: создать user → GET /me → видеть Home System

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Это основной "single source of truth" для frontend. Должен возвращать минимально необходимое.

## 8. Phase 1 — Генерация мира и Home System

Эпик: EPIC-P1-WORLD · Задач: 4

### 8.1. [P1-141] Генерация Home System

|             |                |
|-------------|----------------|
| ID          | P1-141         |
| Эпик        | EPIC-P1-WORLD  |
| Размер      | L              |
| Зависимости | P1-103, P1-102 |

#### Описание

Сервис `generateHomeSystem(userId)`: создаёт систему + 4-7 планет с детерминированным `seed = hash(userId, server_secret)`. Гарантирует, что 1 планета имеет все 5 базовых ресурсов, 1 планета содержит Tritium с низким richness. Все планеты НЕ открыты (`discovered_planets` — пусто, кроме стартовой).

#### Acceptance criteria

- Вызов с одним `userId` всегда возвращает одну и ту же планетарную систему
- 5 базовых ресурсов гарантированно присутствуют
- 1 планета имеет Tritium с richness 0.3-0.8
- Стартовая планета (CommandCenter lvl 1) добавлена в `discovered_planets`
- Tier 3 (кроме Tritium) и Tier 4 ресурсы ОТСУТСТВУЮТ в Home System

#### Файлы для создания / изменения

- `backend/src/features/world/home-system-generator.ts`
- `backend/src/features/world/biomes.ts`
- `backend/src/features/world/home-system-generator.test.ts`

#### Как проверить

Unit-тест: дважды генерируем home system для `userId=1` → результаты идентичны

#### Примечания

Использовать `seedrandom` или собственный детерминированный PRNG. Биомы из GDD §4.2.

#### Готовый per-task prompt (копипаста)

[TASK P1-141] Генерация Home System

**ОПИСАНИЕ**

Сервис `generateHomeSystem(userId)`: создаёт систему + 4-7 планет с детерминированным `seed = hash(userId, server_secret)`. Гарантирует, что 1 планета имеет все 5 базовых ресурсов, 1 планета содержит Tritium с низким richness. Все планеты НЕ открыты (`discovered_planets` — пусто, кроме стартовой).

**ACCEPTANCE CRITERIA**

1. Вызов с одним `userId` всегда возвращает одну и ту же планетарную систему
2. 5 базовых ресурсов гарантированно присутствуют
3. 1 планета имеет Tritium с richness 0.3-0.8
4. Стартовая планета (CommandCenter lvl 1) добавлена в `discovered_planets`
5. Tier 3 (кроме Tritium) и Tier 4 ресурсы **ОТСУТСТВУЮТ** в Home System

**ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ**

- `backend/src/features/world/home-system-generator.ts`
- `backend/src/features/world/biomes.ts`
- `backend/src/features/world/home-system-generator.test.ts`

**ЗАВИСИМОСТИ (должны быть выполнены ранее)**

- P1-103
- P1-102

**КАК ПРОВЕРИТЬ**

Unit-тест: дважды генерируем home system для `userId=1` → результаты идентичны

**ТЕХНИЧЕСКИЕ ЗАМЕТКИ**

Использовать `seedrandom` или собственный детерминированный PRNG. Биомы из GDD §4.2.

## 8.2. [P1-142] Sectors — структура общего пула

|             |               |
|-------------|---------------|
| ID          | P1-142        |
| Эпик        | EPIC-P1-WORLD |
| Размер      | M             |
| Зависимости | P1-103        |

### Описание

Таблица sectors: id (composite x,y,z), seed, generated\_at, system\_count. Сервис getOrCreateSector(x,y,z): возвращает существующий или создаёт пустой. Используется при первом обращении из jump.

### Acceptance criteria

- getOrCreateSector(0,0,0) → создаёт запись если нет
- Повторный вызов возвращает existing
- Seed детерминированный

### Файлы для создания / изменения

- backend/src/db/schema/sectors.ts
- backend/src/features/world/sectors.ts

### Как проверить

Unit-тест: getOrCreateSector(1,2,3) дважды → одна запись в БД

### Примечания

*Размер сектора 500×500×500 (см. addendum §3.7).*

### Готовый per-task prompt (копипаста)

[TASK P1-142] Sectors — структура общего пула

#### ОПИСАНИЕ

Таблица sectors: id (composite x,y,z), seed, generated\_at, system\_count. Сервис getOrCreateSector(x,y,z): возвращает существующий или создаёт пустой. Используется при первом обращении из jump.

#### ACCEPTANCE CRITERIA

1. getOrCreateSector(0,0,0) → создаёт запись если нет
2. Повторный вызов возвращает existing
3. Seed детерминированный

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/db/schema/sectors.ts

- backend/src/features/world/sectors.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-103

КАК ПРОВЕРИТЬ

Unit-тест: getOrCreateSector(1,2,3) дважды → одна запись в БД

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Размер сектора 500×500×500 (см. addendum §3.7).

### 8.3. [P1-143] Lazy generation систем в секторе

|             |                |
|-------------|----------------|
| ID          | P1-143         |
| Эпик        | EPIC-P1-WORLD  |
| Размер      | L              |
| Зависимости | P1-142, P1-141 |

#### Описание

Сервис `generateSystemsInSector(sector, target_count=5)`: создаёт недостающие системы рандомно в координатах сектора. Использует seed сектора. Не превышает 12 систем.

#### Acceptance criteria

- Если в секторе уже 5 систем — ничего не делает
- Если 0 — генерирует 5 систем
- Координаты систем не пересекаются (мин. дистанция 50 ед)
- Биомы планет в системах распределены по весам из GDD

#### Файлы для создания / изменения

- `backend/src/features/world/sector-generator.ts`
- `backend/src/features/world/sector-generator.test.ts`

#### Как проверить

Тест: для пустого сектора генерируется 5 систем; повторный вызов не добавляет

#### Примечания

*Системы в общем пуле — `owner_id=null, is_home_system=false`.*

#### Готовый per-task prompt (копипаста)

[TASK P1-143] Lazy generation систем в секторе

##### ОПИСАНИЕ

Сервис `generateSystemsInSector(sector, target_count=5)`: создаёт недостающие системы рандомно в координатах сектора. Использует seed сектора. Не превышает 12 систем.

##### ACCEPTANCE CRITERIA

1. Если в секторе уже 5 систем — ничего не делает
2. Если 0 — генерирует 5 систем
3. Координаты систем не пересекаются (мин. дистанция 50 ед)
4. Биомы планет в системах распределены по весам из GDD

##### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- `backend/src/features/world/sector-generator.ts`

- backend/src/features/world/sector-generator.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-142
- P1-141

КАК ПРОВЕРИТЬ

Тест: для пустого сектора генерируется 5 систем; повторный вызов не добавляет

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Системы в общем пуле — owner\_id=null, is\_home\_system=false.

### 8.4. [P1-144] Visibility check service

|             |                |
|-------------|----------------|
| ID          | P1-144         |
| Эпик        | EPIC-P1-WORLD  |
| Размер      | M              |
| Зависимости | P1-108, P1-105 |

#### Описание

Сервис checkVisibility(ship): берёт текущую позицию корабля, ищет невидимые игроку планеты/системы в радиусе ship.sensor\_range. Если найдены — добавляет в discovered\_planets/discovered\_systems и возвращает массив новых открытий (для пушей).

#### Acceptance criteria

- Если планета в радиусе и не открыта → попадает в discovered
- Если уже открыта → не дублируется
- Уважается home\_system: чужой корабль не может "увидеть" чужую home system

#### Файлы для создания / изменения

- backend/src/features/world/visibility.ts
- backend/src/features/world/visibility.test.ts

#### Как проверить

Unit-тест: запустить корабль с sensor\_range=30, поставить планету в 25 ед → она открывается

#### Примечания

Радиус — евклидов. Используется в worker tick\_expeditions.

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-144] Visibility check service</p> <p>ОПИСАНИЕ</p> <p>Сервис checkVisibility(ship): берёт текущую позицию корабля, ищет невидимые игроку планеты/системы в радиусе ship.sensor_range. Если найдены — добавляет в discovered_planets/discovered_systems и возвращает массив новых открытий (для пушей).</p> <p>ACCEPTANCE CRITERIA</p> <ol style="list-style-type: none"><li>1. Если планета в радиусе и не открыта → попадает в discovered</li><li>2. Если уже открыта → не дублируется</li><li>3. Уважается home_system: чужой корабль не может "увидеть" чужую home system</li></ol> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- backend/src/features/world/visibility.ts
- backend/src/features/world/visibility.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-108
- P1-105

КАК ПРОВЕРИТЬ

Unit-тест: запустить корабль с `sensor_range=30`, поставить планету в 25 ед → она открывается

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Радиус — евклидов. Используется в `worker tick_expeditions`.

## 9. Phase 1 — Ресурсы и экономика

Эпик: EPIC-P1-RES · Задач: 3

### 9.1. [P1-150] Resource accrual: lazy compute on read

|             |             |
|-------------|-------------|
| ID          | P1-150      |
| Эпик        | EPIC-P1-RES |
| Размер      | M           |
| Зависимости | P1-103      |

#### Описание

Сервис computeCurrentResources(planetId): для каждого resource —  $\text{amount} + \text{rate} \times (\text{now} - \text{last\_update\_at})$ . Не пишет в БД, только возвращает значения. Учитывает cap склада (storage cap).

#### Acceptance criteria

- Если  $\text{last\_update} = \text{now} - 3600\text{s}$ ,  $\text{rate} = 10/\text{h}$  →  $\text{amount} += 10$
- Не превышает storage cap
- Возвращает массив всех ресурсов планеты

#### Файлы для создания / изменения

- backend/src/features/resources/accrual.ts
- backend/src/features/resources/accrual.test.ts

#### Как проверить

Unit-тест с мок-временем

#### Примечания

Это основной паттерн event-based симуляции. Описано в Stellar\_Forge\_GDD.docx §14.3.

#### Готовый per-task prompt (копипаста)

[TASK P1-150] Resource accrual: lazy compute on read

**ОПИСАНИЕ**

Сервис computeCurrentResources(planetId): для каждого resource —  $\text{amount} + \text{rate} \times (\text{now} - \text{last\_update\_at})$ . Не пишет в БД, только возвращает значения. Учитывает cap склада (storage cap).

**ACCEPTANCE CRITERIA**

1. Если  $\text{last\_update} = \text{now} - 3600\text{s}$ ,  $\text{rate} = 10/\text{h}$  →  $\text{amount} += 10$

2. Не превышает storage cap
3. Возвращает массив всех ресурсов планеты

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/resources/accrual.ts
- backend/src/features/resources/accrual.test.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-103

#### КАК ПРОВЕРИТЬ

Unit-тест с мок-временем

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Это основной паттерн event-based симуляции. Описано в Stellar\_Forge\_GDD.docx §14.3.

## 9.2. [P1-151] Resource transactions: spend/gain (атомарно)

|             |             |
|-------------|-------------|
| ID          | P1-151      |
| Эпик        | EPIC-P1-RES |
| Размер      | M           |
| Зависимости | P1-150      |

### Описание

Сервис `spendResources(planetId, costs[])` и `gainResources(planetId, gains[])`: транзакция, `SELECT FOR UPDATE` на `planet_resources`, обновляет `last_update_at`. Возвращает `{success, balanceAfter}` или `{success:false, error:"not enough X"}`.

### Acceptance criteria

- Параллельные вызовы не приводят к отрицательным значениям
- Если не хватает ресурса — транзакция откатывается, ничего не списано
- `last_update_at` синхронизируется со списанием

### Файлы для создания / изменения

- `backend/src/features/resources/transactions.ts`
- `backend/src/features/resources/transactions.test.ts`

### Как проверить

Stress-тест: 100 параллельных списаний → итоговый баланс корректный

### Примечания

*КРИТИЧЕСКАЯ ВЕЩЬ. Любая ошибка тут = дюп ресурсов. Должно быть  $\geq 80\%$  покрытие тестами.*

### Готовый per-task prompt (копипаста)

[TASK P1-151] Resource transactions: spend/gain (атомарно)

#### ОПИСАНИЕ

Сервис `spendResources(planetId, costs[])` и `gainResources(planetId, gains[])`: транзакция, `SELECT FOR UPDATE` на `planet_resources`, обновляет `last_update_at`. Возвращает `{success, balanceAfter}` или `{success:false, error:"not enough X"}`.

#### ACCEPTANCE CRITERIA

1. Параллельные вызовы не приводят к отрицательным значениям
2. Если не хватает ресурса — транзакция откатывается, ничего не списано
3. `last_update_at` синхронизируется со списанием

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/resources/transactions.ts
- backend/src/features/resources/transactions.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-150

КАК ПРОВЕРИТЬ

Stress-тест: 100 параллельных списаний → итоговый баланс корректный

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

КРИТИЧЕСКАЯ ВЕЩЬ. Любая ошибка тут = дюп ресурсов. Должно быть  $\geq 80\%$  покрытие тестами.

### 9.3. [P1-152] POST /resources/convert — Лёд ⇄ Вода

|             |                |
|-------------|----------------|
| ID          | P1-152         |
| Эпик        | EPIC-P1-RES    |
| Размер      | M              |
| Зависимости | P1-151, P1-160 |

#### Описание

Эндпоинт принимает {from: "ice"|"water", to, amount}. Проверяет наличие Cryogenic Factory или Smelter lvl 3+. Списывает ресурс, начисляет другой с учётом потерь (5% при воде → льду, 0% при льду → воде).

#### Acceptance criteria

- Без Cryogenic Factory → 400
- С Cryo lvl 1 → конвертация работает
- Лёд → Вода: 100 льда → 100 воды
- Вода → Лёд: 100 воды → 95 льда
- Энергия списывается

#### Файлы для создания / изменения

- backend/src/features/resources/convert.ts
- backend/src/features/resources/convert.test.ts

#### Как проверить

Integration-тест

#### Примечания

См. addendum §2.1.

#### Готовый per-task prompt (копипаста)

[TASK P1-152] POST /resources/convert — Лёд ⇄ Вода

**ОПИСАНИЕ**

Эндпоинт принимает {from: "ice"|"water", to, amount}. Проверяет наличие Cryogenic Factory или Smelter lvl 3+. Списывает ресурс, начисляет другой с учётом потерь (5% при воде → льду, 0% при льду → воде).

**ACCEPTANCE CRITERIA**

1. Без Cryogenic Factory → 400
2. С Cryo lvl 1 → конвертация работает
3. Лёд → Вода: 100 льда → 100 воды

4. Вода→Лёд: 100 воды → 95 льда
5. Энергия списывается

ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/resources/convert.ts
- backend/src/features/resources/convert.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-151
- P1-160

КАК ПРОВЕРИТЬ

Integration-тест

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

См. addendum §2.1.

## 10. Phase 1 — Здания и постройка

Эпик: EPIC-P1-BLD · Задач: 3

### 10.1. [P1-160] POST /buildings/build — постройка нового здания

|             |                |
|-------------|----------------|
| ID          | P1-160         |
| Эпик        | EPIC-P1-BLD    |
| Размер      | M              |
| Зависимости | P1-151, P1-104 |

#### Описание

Принимает {planetId, typeSlug, slot}. Проверяет: планета принадлежит игроку, есть свободный слот, выполнены deps, хватает ресурсов. Списывает ресурсы, создаёт buildings запись с queue\_completes\_at = now + base\_time, ставит BullMQ job.

#### Acceptance criteria

- Если не хватает ресурсов → 400 с деталями
- Если слот занят → 400
- Если deps не выполнены → 400
- Успех → возвращает building с queue\_completes\_at
- Очередь стройки лимитирована 1 (без премиума)

#### Файлы для создания / изменения

- backend/src/features/buildings/routes.ts
- backend/src/features/buildings/service.ts
- backend/src/features/buildings/service.test.ts

#### Как проверить

Integration-тест полного flow

#### Примечания

Все формулы стоимости из Stellar\_Forge\_GDD.docx §5.2.

#### Готовый per-task prompt (копипаста)

[TASK P1-160] POST /buildings/build — постройка нового здания

##### ОПИСАНИЕ

Принимает {planetId, typeSlug, slot}. Проверяет: планета принадлежит игроку, есть свободный слот, выполнены deps, хватает ресурсов. Списывает ресурсы, создаёт buildings

запись с `queue_completes_at = now + base_time`, ставит BullMQ job.

#### ACCEPTANCE CRITERIA

1. Если не хватает ресурсов → 400 с деталями
2. Если слот занят → 400
3. Если deps не выполнены → 400
4. Успех → возвращает building с `queue_completes_at`
5. Очередь стройки лимитирована 1 (без премиума)

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/buildings/routes.ts
- backend/src/features/buildings/service.ts
- backend/src/features/buildings/service.test.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-151
- P1-104

#### КАК ПРОВЕРИТЬ

Integration-тест полного flow

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Все формулы стоимости из Stellar\_Forge\_GDD.docx §5.2.

## 10.2. [P1-161] POST /buildings/{id}/upgrade — апгрейд здания

|             |             |
|-------------|-------------|
| ID          | P1-161      |
| Эпик        | EPIC-P1-BLD |
| Размер      | M           |
| Зависимости | P1-160      |

### Описание

Аналогично build, но для existing building. Стоимость =  $\text{base\_cost} \times 1.6^{(\text{level}-1)}$ . Время =  $\text{base\_time} \times 1.5^{(\text{level}-1)}$ .

### Acceptance criteria

- Не апгрейдит дальше max\_level
- Стоимость считается по формуле
- queue\_action = "upgrade", level не меняется до завершения

### Файлы для создания / изменения

- backend/src/features/buildings/upgrade.ts
- backend/src/features/buildings/upgrade.test.ts

### Как проверить

Integration-тест

### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-161] POST /buildings/{id}/upgrade — апгрейд здания</p> <p>ОПИСАНИЕ<br/>Аналогично build, но для existing building. Стоимость = <math>\text{base\_cost} \times 1.6^{(\text{level}-1)}</math>. Время = <math>\text{base\_time} \times 1.5^{(\text{level}-1)}</math>.</p> <p>ACCEPTANCE CRITERIA<br/>1. Не апгрейдит дальше max_level<br/>2. Стоимость считается по формуле<br/>3. queue_action = "upgrade", level не меняется до завершения</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br/>- backend/src/features/buildings/upgrade.ts<br/>- backend/src/features/buildings/upgrade.test.ts</p> <p>ЗАВИСИМОСТИ (должны быть выполнены ранее)<br/>- P1-160</p> <p>КАК ПРОВЕРИТЬ<br/>Integration-тест</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

### 10.3. [P1-162] Worker: tick\_buildings

|             |             |
|-------------|-------------|
| ID          | P1-162      |
| Эпик        | EPIC-P1-BLD |
| Размер      | M           |
| Зависимости | P1-161      |

#### Описание

BullMQ воркер, бежит каждые 30 секунд. Находит buildings, у которых `queue_completes_at ≤ now`. Применяет результат: для build — `level=1`, для upgrade — `level+=1`. Обновляет `regen_rate` ресурсов на этой планете (в `planet_resources`).

#### Acceptance criteria

- Здание завершается ровно по таймеру ( $\pm 30$  сек)
- Скорость добычи на планете обновляется (если это шахта)
- Уведомление кладётся в notifications таблицу

#### Файлы для создания / изменения

- backend/src/workers/tick-buildings.ts
- backend/src/workers/tick-buildings.test.ts

#### Как проверить

Integration-тест: построить здание с `base_time=10s`, подождать → `level=1`

#### Примечания

Использовать BullMQ *repeat-job*.

#### Готовый per-task prompt (копипаста)

[TASK P1-162] Worker: tick\_buildings

##### ОПИСАНИЕ

BullMQ воркер, бежит каждые 30 секунд. Находит buildings, у которых `queue_completes_at ≤ now`. Применяет результат: для build — `level=1`, для upgrade — `level+=1`. Обновляет `regen_rate` ресурсов на этой планете (в `planet_resources`).

##### ACCEPTANCE CRITERIA

1. Здание завершается ровно по таймеру ( $\pm 30$  сек)
2. Скорость добычи на планете обновляется (если это шахта)
3. Уведомление кладётся в notifications таблицу

##### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/workers/tick-buildings.ts

- backend/src/workers/tick-buildings.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-161

КАК ПРОВЕРИТЬ

Integration-тест: построить здание с base\_time=10s, подождать → level=1

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать BullMQ repeat-job.

# 11. Phase 1 — Корабли и крафт

Эпик: EPIC-P1-SHP · Задач: 2

## 11.1. [P1-170] POST /ships/build — постройка корабля

|             |                |
|-------------|----------------|
| ID          | P1-170         |
| Эпик        | EPIC-P1-SHP    |
| Размер      | M              |
| Зависимости | P1-151, P1-105 |

### Описание

Принимает {planetId, typeSlug}. Проверяет: shipyard есть, deps выполнены, ресурсы хватает. Списывает, ставит в очередь верфи. Очередь лимитирована.

### Acceptance criteria

- Без shipyard → 400
- Слот верфи занят → 400
- Успех → возвращает ship с status="building"

### Файлы для создания / изменения

- backend/src/features/ships/build.ts
- backend/src/features/ships/build.test.ts

### Как проверить

Integration-тест

### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-170] POST /ships/build — постройка корабля</p> <p>ОПИСАНИЕ<br/>Принимает {planetId, typeSlug}. Проверяет: shipyard есть, deps выполнены, ресурсы хватает. Списывает, ставит в очередь верфи. Очередь лимитирована.</p> <p>ACCEPTANCE CRITERIA<br/>1. Без shipyard → 400<br/>2. Слот верфи занят → 400<br/>3. Успех → возвращает ship с status="building"</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br/>- backend/src/features/ships/build.ts<br/>- backend/src/features/ships/build.test.ts</p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-151
- P1-105

КАК ПРОВЕРИТЬ

Integration-тест

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 11.2. [P1-171] Worker: tick\_ships

|             |             |
|-------------|-------------|
| ID          | P1-171      |
| Эпик        | EPIC-P1-SHP |
| Размер      | M           |
| Зависимости | P1-170      |

### Описание

BullMQ воркер. Завершает постройку кораблей. Меняет status: building → idle, location\_planet\_id = where\_built.

### Acceptance criteria

- Корабль готов в срок
- Уведомление в notifications

### Файлы для создания / изменения

- backend/src/workers/tick-ships.ts

### Как проверить

Integration-тест

### Готовый per-task prompt (копипаста)

[TASK P1-171] Worker: tick\_ships

#### ОПИСАНИЕ

BullMQ воркер. Завершает постройку кораблей. Меняет status: building → idle, location\_planet\_id = where\_built.

#### ACCEPTANCE CRITERIA

1. Корабль готов в срок
2. Уведомление в notifications

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/workers/tick-ships.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-170

#### КАК ПРОВЕРИТЬ

Integration-тест

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 12. Phase 1 — Экспедиции и видимость

Эпик: EPIC-P1-EXP · Задач: 3

### 12.1. [P1-180] POST /expeditions — отправить корабль

|             |                |
|-------------|----------------|
| ID          | P1-180         |
| Эпик        | EPIC-P1-EXP    |
| Размер      | L              |
| Зависимости | P1-170, P1-151 |

#### Описание

Принимает {shipId, targetX, targetY, targetZ, fuelLoaded, cargoLoaded}. Проверяет: корабль idle, принадлежит игроку, на стартовой планете есть достаточно топлива и груза. Создаёт expedition, ставит ship.status=moving, рассчитывает eta.

#### Acceptance criteria

- Корабль не idle → 400
- Не хватает топлива → 400
- $\text{eta} = \text{distance} \times 60 / \text{speed} \times \text{engine\_factor}$
- BullMQ job на eta запланирован

#### Файлы для создания / изменения

- backend/src/features/expeditions/launch.ts
- backend/src/features/expeditions/launch.test.ts

#### Как проверить

Integration-тест: запустить scout на 10 ед → видеть expedition с eta

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-180] POST /expeditions — отправить корабль</p> <p>ОПИСАНИЕ</p> <p>Принимает {shipId, targetX, targetY, targetZ, fuelLoaded, cargoLoaded}. Проверяет: корабль idle, принадлежит игроку, на стартовой планете есть достаточно топлива и груза. Создаёт expedition, ставит ship.status=moving, рассчитывает eta.</p> <p>ACCEPTANCE CRITERIA</p> <ol style="list-style-type: none"><li>1. Корабль не idle → 400</li><li>2. Не хватает топлива → 400</li><li>3. <math>\text{eta} = \text{distance} \times 60 / \text{speed} \times \text{engine\_factor}</math></li><li>4. BullMQ job на eta запланирован</li></ol> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/expeditions/launch.ts
- backend/src/features/expeditions/launch.test.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-170
- P1-151

#### КАК ПРОВЕРИТЬ

Integration-тест: запустить scout на 10 ед → видеть expedition с eta

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 12.2. [P1-181] Worker: tick\_expeditions

|             |                |
|-------------|----------------|
| ID          | P1-181         |
| Эпик        | EPIC-P1-EXP    |
| Размер      | L              |
| Зависимости | P1-180, P1-144 |

### Описание

BullMQ воркер каждые 30 сек. Для всех `in_flight expeditions`: рассчитывает текущую позицию (`linear interp`), вызывает `checkVisibility`, если `eta` достигнут — переводит в "arrived" и стартует возврат (если `scout/probe`). При возврате — `apply` результат (`planet discovery`, `cargo unload`).

### Acceptance criteria

- Корабль "движется" во времени, видимость обновляется
- По прибытии — `discovered_planets` обновляется (если `visibility` сработал)
- Возвращается в стартовую планету (для `scout/cargo-empty`), `eta` удваивается

### Файлы для создания / изменения

- `backend/src/workers/tick-expeditions.ts`
- `backend/src/workers/tick-expeditions.test.ts`

### Как проверить

Integration-тест полного цикла `scout` → `return`

### Примечания

Самая сложная логика. Тестировать каждый кейс отдельно.

### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-181] Worker: tick_expeditions</p> <p>ОПИСАНИЕ</p> <p>BullMQ воркер каждые 30 сек. Для всех <code>in_flight expeditions</code>: рассчитывает текущую позицию (<code>linear interp</code>), вызывает <code>checkVisibility</code>, если <code>eta</code> достигнут — переводит в "arrived" и стартует возврат (если <code>scout/probe</code>). При возврате — <code>apply</code> результат (<code>planet discovery</code>, <code>cargo unload</code>).</p> <p>ACCEPTANCE CRITERIA</p> <ol style="list-style-type: none"><li>1. Корабль "движется" во времени, видимость обновляется</li><li>2. По прибытии — <code>discovered_planets</code> обновляется (если <code>visibility</code> сработал)</li><li>3. Возвращается в стартовую планету (для <code>scout/cargo-empty</code>), <code>eta</code> удваивается</li></ol> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ</p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- backend/src/workers/tick-expeditions.ts
- backend/src/workers/tick-expeditions.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-180
- P1-144

КАК ПРОВЕРИТЬ

Integration-тест полного цикла scout → return

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Самая сложная логика. Тестировать каждый кейс отдельно.

### 12.3. [P1-182] POST /expeditions/jump — межсистемный прыжок

|             |                |
|-------------|----------------|
| ID          | P1-182         |
| Эпик        | EPIC-P1-EXP    |
| Размер      | M              |
| Зависимости | P1-180, P1-143 |

#### Описание

Особый эндпоинт для Jump Ship. Принимает {shipId, targetSector{x,y,z}}. Проверяет: ship.type = jump\_ship, есть Jump Fuel в баке, Jump Drive исследован lvl 1+. Lazy-генерит сектор, выбирает целевую систему. После прибытия — добавляет в discovered\_systems.

#### Acceptance criteria

- Без Jump Drive → 400
- Не Jump Ship → 400
- Не хватает Jump Fuel → 400
- Успех → корабль попадает в общий пул

#### Файлы для создания / изменения

- backend/src/features/expeditions/jump.ts

#### Как проверить

Integration-тест

#### Примечания

См. addendum §3.5-§3.6.

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-182] POST /expeditions/jump — межсистемный прыжок</p> <p>ОПИСАНИЕ</p> <p>Особый эндпоинт для Jump Ship. Принимает {shipId, targetSector{x,y,z}}. Проверяет: ship.type = jump_ship, есть Jump Fuel в баке, Jump Drive исследован lvl 1+. Lazy-генерит сектор, выбирает целевую систему. После прибытия — добавляет в discovered_systems.</p> <p>ACCEPTANCE CRITERIA</p> <ol style="list-style-type: none"><li>1. Без Jump Drive → 400</li><li>2. Не Jump Ship → 400</li><li>3. Не хватает Jump Fuel → 400</li><li>4. Успех → корабль попадает в общий пул</li></ol> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- backend/src/features/expeditions/jump.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-180
- P1-143

КАК ПРОВЕРИТЬ  
Integration-тест

ТЕХНИЧЕСКИЕ ЗАМЕТКИ  
См. addendum §3.5-§3.6.

## 13. Phase 1 — Frontend (ТМА)

Эпик: EPIC-P1-FE · Задач: 7

### 13.1. [P1-200] API client + auth flow

|             |                |
|-------------|----------------|
| ID          | P1-200         |
| Эпик        | EPIC-P1-FE     |
| Размер      | M              |
| Зависимости | P0-004, P1-122 |

#### Описание

frontend/src/lib/api.ts: единый клиент с автоматической передачей session token из Telegram WebApp SDK. React Query для всех запросов. Auth flow: при загрузке → POST /auth/telegram → сохранить token → GET /me.

#### Acceptance criteria

- При первом запуске игрок авторизован автоматически
- Token хранится в memory (HE localStorage)
- Все запросы автоматически шлют Authorization header

#### Файлы для создания / изменения

- frontend/src/lib/api.ts
- frontend/src/hooks/useAuth.ts
- frontend/src/hooks/useMe.ts

#### Как проверить

Открыть ТМА → видеть данные игрока

#### Примечания

Использовать @tanstack/react-query.

#### Готовый per-task prompt (копипаста)

[TASK P1-200] API client + auth flow

**ОПИСАНИЕ**

frontend/src/lib/api.ts: единый клиент с автоматической передачей session token из Telegram WebApp SDK. React Query для всех запросов. Auth flow: при загрузке → POST /auth/telegram → сохранить token → GET /me.

**ACCEPTANCE CRITERIA**

1. При первом запуске игрок авторизован автоматически
2. Token хранится в memory (НЕ localStorage)
3. Все запросы автоматически шлют Authorization header

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- frontend/src/lib/api.ts
- frontend/src/hooks/useAuth.ts
- frontend/src/hooks/useMe.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P0-004
- P1-122

#### КАК ПРОВЕРИТЬ

Открыть ТМА → видеть данные игрока

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать @tanstack/react-query.

### 13.2. [P1-201] Главный экран: ресурсы + очередь стройки

|             |            |
|-------------|------------|
| ID          | P1-201     |
| Эпик        | EPIC-P1-FE |
| Размер      | L          |
| Зависимости | P1-200     |

#### Описание

Основной экран ТМА: верх — плашка с балансом ресурсов (с регенерацией в реальном времени через requestAnimationFrame), середина — текущая фокус-планета (схема со зданиями), низ — таб-бар: Planets, Ships, Map, Tech, Profile.

#### Acceptance criteria

- Ресурсы тикают в UI (анимированно увеличиваются)
- Очередь стройки видна с countdown
- Тар на здание → открывает деталь

#### Файлы для создания / изменения

- frontend/src/pages/Home.tsx
- frontend/src/components/ResourceBar.tsx
- frontend/src/components/PlanetView.tsx
- frontend/src/components/BuildQueue.tsx

#### Как проверить

UI matches описание

#### Примечания

Тёмная тема, космический стиль. Tailwind + Lucide icons.

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-201] Главный экран: ресурсы + очередь стройки</p> <p>ОПИСАНИЕ</p> <p>Основной экран ТМА: верх — плашка с балансом ресурсов (с регенерацией в реальном времени через requestAnimationFrame), середина — текущая фокус-планета (схема со зданиями), низ — таб-бар: Planets, Ships, Map, Tech, Profile.</p> <p>ACCEPTANCE CRITERIA</p> <ol style="list-style-type: none"><li>1. Ресурсы тикают в UI (анимированно увеличиваются)</li><li>2. Очередь стройки видна с countdown</li><li>3. Тар на здание → открывает деталь</li></ol> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- frontend/src/pages/Home.tsx
- frontend/src/components/ResourceBar.tsx
- frontend/src/components/PlanetView.tsx
- frontend/src/components/BuildQueue.tsx

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-200

#### КАК ПРОВЕРИТЬ

UI matches описание

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Тёмная тема, космический стиль. Tailwind + Lucide icons.

### 13.3. [P1-202] Экран планеты: слоты и апгрейды

|             |                        |
|-------------|------------------------|
| ID          | P1-202                 |
| Эпик        | EPIC-P1-FE             |
| Размер      | M                      |
| Зависимости | P1-201, P1-160, P1-161 |

#### Описание

Список зданий планеты + пустые слоты. Тар на слот → диалог выбора здания. Тар на existing → диалог апгрейда с показом стоимости.

#### Acceptance criteria

- Видны все здания + пустые слоты
- Стоимость апгрейда видна заранее
- После постройки/апгрейда — оптимистичный UI с rollback при ошибке

#### Файлы для создания / изменения

- frontend/src/pages/PlanetDetail.tsx
- frontend/src/components/BuildingSlot.tsx
- frontend/src/components/UpgradeDialog.tsx

#### Как проверить

Полный flow стройки шахты

#### Готовый per-task prompt (копипаста)

[TASK P1-202] Экран планеты: слоты и апгрейды

**ОПИСАНИЕ**

Список зданий планеты + пустые слоты. Тар на слот → диалог выбора здания. Тар на existing → диалог апгрейда с показом стоимости.

**ACCEPTANCE CRITERIA**

1. Видны все здания + пустые слоты
2. Стоимость апгрейда видна заранее
3. После постройки/апгрейда — оптимистичный UI с rollback при ошибке

**ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ**

- frontend/src/pages/PlanetDetail.tsx
- frontend/src/components/BuildingSlot.tsx
- frontend/src/components/UpgradeDialog.tsx

**ЗАВИСИМОСТИ (должны быть выполнены ранее)**

- P1-201

- P1-160
- P1-161

КАК ПРОВЕРИТЬ  
Полный flow стройки шахты

ТЕХНИЧЕСКИЕ ЗАМЕТКИ  
—

### 13.4. [P1-203] Карта домашней системы

|             |            |
|-------------|------------|
| ID          | P1-203     |
| Эпик        | EPIC-P1-FE |
| Размер      | M          |
| Зависимости | P1-201     |

#### Описание

Top-down карта Home System на PIXIJS. Звезда в центре, планеты на орбитах, открытые — с именами и ресурсами, неоткрытые — "?". Маркеры кораблей с траекториями.

#### Acceptance criteria

- Карта рендерится 60fps на mid-tier мобильных
- Pan + pinch zoom
- Тар на планету → переход на экран планеты

#### Файлы для создания / изменения

- frontend/src/pages/SystemMap.tsx
- frontend/src/components/pixi/SystemRenderer.tsx

#### Как проверить

Открыть карту → видеть свою систему

#### Примечания

PIXIJS v8.

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-203] Карта домашней системы</p> <p>ОПИСАНИЕ</p> <p>Top-down карта Home System на PIXIJS. Звезда в центре, планеты на орбитах, открытые — с именами и ресурсами, неоткрытые — "?". Маркеры кораблей с траекториями.</p> <p>ACCEPTANCE CRITERIA</p> <p>1. Карта рендерится 60fps на mid-tier мобильных</p> <p>2. Pan + pinch zoom</p> <p>3. Тар на планету → переход на экран планеты</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ</p> <p>- frontend/src/pages/SystemMap.tsx</p> <p>- frontend/src/components/pixi/SystemRenderer.tsx</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

ЗАВИСИМОСТИ (должны быть выполнены ранее)  
- P1-201

КАК ПРОВЕРИТЬ

Открыть карту → видеть свою систему

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

PixiJS v8.

## 13.5. [P1-204] Экран кораблей + запуск экспедиции

|             |                |
|-------------|----------------|
| ID          | P1-204         |
| Эпик        | EPIC-P1-FE     |
| Размер      | M              |
| Зависимости | P1-203, P1-180 |

### Описание

Список кораблей с состоянием. Кнопка «Отправить» → диалог: выбор точки на мини-карте + загрузка топлива + ETA preview.

### Acceptance criteria

- Видно все корабли
- Можно запустить scout, видеть его на карте
- ETA отсчитывается в реальном времени

### Файлы для создания / изменения

- frontend/src/pages/Ships.tsx
- frontend/src/components/ExpeditionDialog.tsx

### Как проверить

Запустить scout → видеть в полёте на карте

### Готовый per-task prompt (копипаста)

[TASK P1-204] Экран кораблей + запуск экспедиции

#### ОПИСАНИЕ

Список кораблей с состоянием. Кнопка «Отправить» → диалог: выбор точки на мини-карте + загрузка топлива + ETA preview.

#### ACCEPTANCE CRITERIA

1. Видно все корабли
2. Можно запустить scout, видеть его на карте
3. ETA отсчитывается в реальном времени

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- frontend/src/pages/Ships.tsx
- frontend/src/components/ExpeditionDialog.tsx

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-203
- P1-180

#### КАК ПРОВЕРИТЬ

Запустить scout → видеть в полёте на карте

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

### 13.6. [P1-205] Экран исследований (tech tree)

|             |                |
|-------------|----------------|
| ID          | P1-205         |
| Эпик        | EPIC-P1-FE     |
| Размер      | M              |
| Зависимости | P1-200, P1-106 |

#### Описание

7 веток × 5 уровней. Тар на узел → диалог с деталями + кнопка start. Отображение прогресса.

#### Acceptance criteria

- Все ветки видны
- Можно начать исследование если хватает ресурсов и lab lvl
- Прогресс таймера в реальном времени

#### Файлы для создания / изменения

- frontend/src/pages/Research.tsx
- frontend/src/components/TechTreeNode.tsx

#### Как проверить

Start research → ждать → done

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-205] Экран исследований (tech tree)</p> <p>ОПИСАНИЕ</p> <p>7 веток × 5 уровней. Тар на узел → диалог с деталями + кнопка start. Отображение прогресса.</p> <p>ACCEPTANCE CRITERIA</p> <p>1. Все ветки видны</p> <p>2. Можно начать исследование если хватает ресурсов и lab lvl</p> <p>3. Прогресс таймера в реальном времени</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ</p> <p>- frontend/src/pages/Research.tsx</p> <p>- frontend/src/components/TechTreeNode.tsx</p> <p>ЗАВИСИМОСТИ (должны быть выполнены ранее)</p> <p>- P1-200</p> <p>- P1-106</p> <p>КАК ПРОВЕРИТЬ</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Start research → ждать → done

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

### 13.7. [P1-206] Onboarding tutorial

|             |                        |
|-------------|------------------------|
| ID          | P1-206                 |
| Эпик        | EPIC-P1-FE             |
| Размер      | M                      |
| Зависимости | P1-201, P1-202, P1-204 |

#### Описание

5 шагов: Welcome → Build first mine → Build storage → Build scout → Send first expedition.  
Skipable, но ресурсы за выполнение.

#### Acceptance criteria

- Запускается на первом входе нового user
- Можно скипнуть и вернуться позже
- Награда (200 Fe + 100 H<sub>2</sub>O) за прохождение

#### Файлы для создания / изменения

- frontend/src/components/Tutorial.tsx
- frontend/src/pages/onboarding/

#### Как проверить

Создать нового user → пройти tutorial → получить награду

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-206] Onboarding tutorial</p> <p>ОПИСАНИЕ</p> <p>5 шагов: Welcome → Build first mine → Build storage → Build scout → Send first expedition.<br/>Skipable, но ресурсы за выполнение.</p> <p>ACCEPTANCE CRITERIA</p> <p>1. Запускается на первом входе нового user</p> <p>2. Можно скипнуть и вернуться позже</p> <p>3. Награда (200 Fe + 100 H<sub>2</sub>O) за прохождение</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ</p> <p>- frontend/src/components/Tutorial.tsx</p> <p>- frontend/src/pages/onboarding/</p> <p>ЗАВИСИМОСТИ (должны быть выполнены ранее)</p> <p>- P1-201</p> <p>- P1-202</p> <p>- P1-204</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### КАК ПРОВЕРИТЬ

Создать нового user → пройти tutorial → получить награду

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

## 14. Phase 1 — Telegram-бот и пуши

Эпик: EPIC-P1-BOT · Задач: 2

### 14.1. [P1-220] Telegram bot webhook + /start

|             |             |
|-------------|-------------|
| ID          | P1-220      |
| Эпик        | EPIC-P1-BOT |
| Размер      | M           |
| Зависимости | P0-010      |

#### Описание

POST /tg/webhook принимает обновления от Telegram. Команда /start отвечает с inline-кнопкой "Открыть New Universe" → запуск Mini App.

#### Acceptance criteria

- Webhook настроен
- /start работает
- Кнопка открывает ТМА

#### Файлы для создания / изменения

- backend/src/features/bot/webhook.ts
- backend/src/features/bot/commands.ts

#### Как проверить

В Telegram: /start → видеть кнопку

#### Готовый per-task prompt (копипаста)

[TASK P1-220] Telegram bot webhook + /start

##### ОПИСАНИЕ

POST /tg/webhook принимает обновления от Telegram. Команда /start отвечает с inline-кнопкой "Открыть New Universe" → запуск Mini App.

##### ACCEPTANCE CRITERIA

1. Webhook настроен
2. /start работает
3. Кнопка открывает ТМА

##### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/features/bot/webhook.ts
- backend/src/features/bot/commands.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)  
- P0-010

КАК ПРОВЕРИТЬ  
В Telegram: /start → видеть кнопку

ТЕХНИЧЕСКИЕ ЗАМЕТКИ  
—

## 14.2. [P1-221] Push-уведомления о ключевых событиях

|             |                |
|-------------|----------------|
| ID          | P1-221         |
| Эпик        | EPIC-P1-BOT    |
| Размер      | M              |
| Зависимости | P1-220, P1-162 |

### Описание

Сервис sendPush(userId, type, payload). Воркер раз в минуту берёт notifications из БД с pending=true и шлёт в Telegram. Для типов: building\_done, ship\_done, expedition\_returned, research\_done.

### Acceptance criteria

- После завершения постройки → пуш приходит в Telegram
- Notification помечается sent\_at
- Соблюдается лимит 20 сообщений/мин на user

### Файлы для создания / изменения

- backend/src/workers/notifications.ts
- backend/src/features/bot/push.ts

### Как проверить

Построить здание → дождаться → пуш в Telegram

### Готовый per-task prompt (копипаста)

[TASK P1-221] Push-уведомления о ключевых событиях

#### ОПИСАНИЕ

Сервис sendPush(userId, type, payload). Воркер раз в минуту берёт notifications из БД с pending=true и шлёт в Telegram. Для типов: building\_done, ship\_done, expedition\_returned, research\_done.

#### ACCEPTANCE CRITERIA

1. После завершения постройки → пуш приходит в Telegram
2. Notification помечается sent\_at
3. Соблюдается лимит 20 сообщений/мин на user

#### ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/src/workers/notifications.ts
- backend/src/features/bot/push.ts

#### ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-220
- P1-162

#### КАК ПРОВЕРИТЬ

Построить здание → дожидаться → пуш в Telegram

#### ТЕХНИЧЕСКИЕ ЗАМЕТКИ

—

# 15. Phase 1 — Тесты и QA

Эпик: EPIC-P1-QA · Задач: 2

## 15.1. [P1-240] Integration test: full first-day flow

|             |                        |
|-------------|------------------------|
| ID          | P1-240                 |
| Эпик        | EPIC-P1-QA             |
| Размер      | L                      |
| Зависимости | P1-181, P1-162, P1-171 |

### Описание

Один большой E2E-тест: создать user → home system сгенерирована → построить шахту → апгрейднуть → построить scout → запустить → дожждаться возврата → открыть планету.

### Acceptance criteria

- Тест проходит за < 30 сек (с ускоренным временем)
- Покрывает основные ошибочные пути

### Файлы для создания / изменения

- backend/tests/e2e/first-day.test.ts

### Как проверить

`npm run test:e2e` зелёный

### Примечания

Использовать testcontainers если нужен изолированный Postgres.

### Готовый per-task prompt (копипаста)

[TASK P1-240] Integration test: full first-day flow

ОПИСАНИЕ

Один большой E2E-тест: создать user → home system сгенерирована → построить шахту → апгрейднуть → построить scout → запустить → дожждаться возврата → открыть планету.

ACCEPTANCE CRITERIA

1. Тест проходит за < 30 сек (с ускоренным временем)

2. Покрывает основные ошибочные пути

ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ

- backend/tests/e2e/first-day.test.ts

ЗАВИСИМОСТИ (должны быть выполнены ранее)

- P1-181
- P1-162
- P1-171

КАК ПРОВЕРИТЬ

``npm run test:e2e`` зелёный

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Использовать testcontainers если нужен изолированный Postgres.

## 15.2. [P1-241] Playwright E2E на frontend

|             |            |
|-------------|------------|
| ID          | P1-241     |
| Эпик        | EPIC-P1-QA |
| Размер      | M          |
| Зависимости | P1-206     |

### Описание

Тест: открыть ТМА с моком Telegram initData → пройти onboarding → построить шахту → проверить что ресурсы тикают.

### Acceptance criteria

- Тест проходит

### Файлы для создания / изменения

- frontend/tests/e2e/onboarding.spec.ts

### Как проверить

`npx playwright test` зелёный

### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P1-241] Playwright E2E на frontend</p> <p>ОПИСАНИЕ<br/>Тест: открыть ТМА с моком Telegram initData → пройти onboarding → построить шахту → проверить что ресурсы тикают.</p> <p>ACCEPTANCE CRITERIA<br/>1. Тест проходит</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br/>- frontend/tests/e2e/onboarding.spec.ts</p> <p>ЗАВИСИМОСТИ (должны быть выполнены ранее)<br/>- P1-206</p> <p>КАК ПРОВЕРИТЬ<br/>`npx playwright test` зелёный</p> <p>ТЕХНИЧЕСКИЕ ЗАМЕТКИ<br/>—</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 16. Phase 2 — Outline (Expansion)

Эпик: EPIC-P2-OUT · Задач: 3

### 16.1. [P2-EPIC-COLONIZE] [Эпик] Колонизация и вторая планета

|             |                  |
|-------------|------------------|
| ID          | P2-EPIC-COLONIZE |
| Эпик        | EPIC-P2-OUT      |
| Размер      | XL               |
| Зависимости | P1-181           |

#### Описание

Колонизатор как корабль, развёртывание новой колонии в Common Pool, синхронизация ресурсов между планетами через Cargo.

#### Acceptance criteria

- К концу Phase 2 игрок может иметь 2-3 колонии и перевозить груз

#### Файлы для создания / изменения

- будут расписаны при разбиении эпика на микротаски

#### Как проверить

—

#### Примечания

Разбить на ~10 микротасок при старте Phase 2.

#### Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P2-EPIC-COLONIZE] [Эпик] Колонизация и вторая планета</p> <p>ОПИСАНИЕ<br/>Колонизатор как корабль, развёртывание новой колонии в Common Pool, синхронизация ресурсов между планетами через Cargo.</p> <p>ACCEPTANCE CRITERIA<br/>1. К концу Phase 2 игрок может иметь 2-3 колонии и перевозить груз</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br/>- будут расписаны при разбиении эпика на микротаски</p> <p>ЗАВИСИМОСТИ (должны быть выполнены ранее)<br/>- P1-181</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

КАК ПРОВЕРИТЬ

—

ТЕХНИЧЕСКИЕ ЗАМЕТКИ

Разбить на ~10 микротасок при старте Phase 2.

16.2. [P2-EPIC-MARKET-NPC] [Эпик] NPC-маркет (биржа)

|             |                    |
|-------------|--------------------|
| ID          | P2-EPIC-MARKET-NPC |
| Эпик        | EPIC-P2-OUT        |
| Размер      | L                  |
| Зависимости | P2-EPIC-COLONIZE   |

Описание

Игрок может выставять и покупать ордера у NPC брокера в системе. Карго-перевозка автоматизированно.

Acceptance criteria

- Эконом-цикл: продать излишек одного ресурса за нужный

Файлы для создания / изменения

- —

Как проверить

—

Готовый per-task prompt (копипаста)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>[TASK P2-EPIC-MARKET-NPC] [Эпик] NPC-маркет (биржа)</p> <p>ОПИСАНИЕ<br/>Игрок может выставять и покупать ордера у NPC брокера в системе. Карго-перевозка автоматизированно.</p> <p>АССЕРТАНЦЕ CRITERIA<br/>1. Эконом-цикл: продать излишек одного ресурса за нужный</p> <p>ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br/>- —</p> <p>ЗАВИСИМОСТИ (должны быть выполнены ранее)<br/>- P2-EPIC-COLONIZE</p> <p>КАК ПРОВЕРИТЬ<br/>—</p> <p>ТЕХНИЧЕСКИЕ ЗАМЕТКИ<br/>—</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

16.3. [P2-EPIC-RESEARCH] [Эпик] Research lvl 1-3 для всех веток

|             |                  |
|-------------|------------------|
| ID          | P2-EPIC-RESEARCH |
| Эпик        | EPIC-P2-OUT      |
| Размер      | L                |
| Зависимости | P1-205           |

Описание

Полная имплементация tech-веток до 3 уровня + UI прогресса.

Acceptance criteria

- Игрок может пройти любую ветку до lvl 3

Файлы для создания / изменения

- —

Как проверить

—

Готовый per-task prompt (копипаста)

|                                                                         |
|-------------------------------------------------------------------------|
| [TASK P2-EPIC-RESEARCH] [Эпик] Research lvl 1-3 для всех веток          |
| ОПИСАНИЕ<br>Полная имплементация tech-веток до 3 уровня + UI прогресса. |
| АССЕРТАНЦЕ CRITERIA<br>1. Игрок может пройти любую ветку до lvl 3       |
| ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br>- —                                     |
| ЗАВИСИМОСТИ (должны быть выполнены ранее)<br>- P1-205                   |
| КАК ПРОВЕРИТЬ<br>—                                                      |
| ТЕХНИЧЕСКИЕ ЗАМЕТКИ<br>—                                                |

## 17. Phase 3 — Outline (Multiplayer)

Эпик: EPIC-P3-OUT · Задач: 3

### 17.1. [P3-EPIC-MULTIPLAYER-MAP] [Эпик] Multiplayer карта Common Pool

|             |                         |
|-------------|-------------------------|
| ID          | P3-EPIC-MULTIPLAYER-MAP |
| Эпик        | EPIC-P3-OUT             |
| Размер      | XL                      |
| Зависимости | P1-182                  |

#### Описание

Видны другие игроки, их колонии, флоты. Глобальный поиск систем. Sector map UI.

#### Acceptance criteria

- Игрок видит соседей в Sector Map

#### Файлы для создания / изменения

- —

#### Как проверить

—

#### Готовый per-task prompt (копипаста)

|                                                                                             |
|---------------------------------------------------------------------------------------------|
| [TASK P3-EPIC-MULTIPLAYER-MAP] [Эпик] Multiplayer карта Common Pool                         |
| ОПИСАНИЕ<br>Видны другие игроки, их колонии, флоты. Глобальный поиск систем. Sector map UI. |
| АССЕРТАНСЕ КРИТЕРИА<br>1. Игрок видит соседей в Sector Map                                  |
| ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br>- —                                                         |
| ЗАВИСИМОСТИ (должны быть выполнены ранее)<br>- P1-182                                       |
| КАК ПРОВЕРИТЬ<br>—                                                                          |
| ТЕХНИЧЕСКИЕ ЗАМЕТКИ                                                                         |



17.2. [P3-EPIC-ALLIANCES] [Эпик] Альянсы (кланы)

|             |                         |
|-------------|-------------------------|
| ID          | P3-EPIC-ALLIANCES       |
| Эпик        | EPIC-P3-OUT             |
| Размер      | L                       |
| Зависимости | P3-EPIC-MULTIPLAYER-MAP |

Описание

Создание альянса, общий банк, чат, карта членов, объявления о войне.

Acceptance criteria

- Игроки могут объединяться в альянсы до 30 человек

Файлы для создания / изменения

- —

Как проверить

—

Готовый per-task prompt (копипаста)

|                                                                                  |
|----------------------------------------------------------------------------------|
| [TASK P3-EPIC-ALLIANCES] [Эпик] Альянсы (кланы)                                  |
| ОПИСАНИЕ<br>Создание альянса, общий банк, чат, карта членов, объявления о войне. |
| АССЕРТАНЦЕ CRITERIA<br>1. Игроки могут объединяться в альянсы до 30 человек      |
| ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ<br>- —                                              |
| ЗАВИСИМОСТИ (должны быть выполнены ранее)<br>- P3-EPIC-MULTIPLAYER-MAP           |
| КАК ПРОВЕРИТЬ<br>—                                                               |
| ТЕХНИЧЕСКИЕ ЗАМЕТКИ<br>—                                                         |

17.3. [P3-EPIC-MARKET-PVP] [Эпик] Маркет между игроками

|             |                    |
|-------------|--------------------|
| ID          | P3-EPIC-MARKET-PVP |
| Эпик        | EPIC-P3-OUT        |
| Размер      | L                  |
| Зависимости | P2-EPIC-MARKET-NPC |

Описание

Биржа доступна между игроками с автоматической доставкой через NPC-карго.

Acceptance criteria

- Игроки могут торговать друг с другом

Файлы для создания / изменения

- —

Как проверить

—

Готовый per-task prompt (копипаста)

[TASK P3-EPIC-MARKET-PVP] [Эпик] Маркет между игроками

ОПИСАНИЕ  
Биржа доступна между игроками с автоматической доставкой через NPC-карго.

АССЕРТАНЦЕ CRITERIA  
1. Игроки могут торговать друг с другом

ФАЙЛЫ ДЛЯ СОЗДАНИЯ/ИЗМЕНЕНИЯ  
- —

ЗАВИСИМОСТИ (должны быть выполнены ранее)  
- P2-EPIC-MARKET-NPC

КАК ПРОВЕРИТЬ  
—

ТЕХНИЧЕСКИЕ ЗАМЕТКИ  
—

## 18. Итоги и следующие шаги

В Phase 0 + Phase 1 расписано 47 конкретных микротасок, плюс 6 эпиков-заголовков для Phase 2-3 (разбиваются позже).

### Стратегия запуска

1. Импортируй задачи в GitHub Project (см. [tasks/README\\_github\\_import.md](#)).
2. Возьми P0-001..P0-005 как первый спринт — это setup на 1-2 дня.
3. Параллельно делай P1-101..P1-108 (схема БД) — фундамент игры.
4. Для каждой задачи передавай в исполняющую модель: глобальный system prompt + per-task prompt из этого документа. Дополнительно прикрепляй ссылки на GDD и addendum.
5. После закрытия группы задач — прогон тестов и интеграционный смок.